

Санкт-Петербургский национальный
исследовательский институт информационных
технологий, механики и оптики

Физический факультет



Практическая работа №3

**Разработка реверсивного 8-разрядного двоичного
счётчика по задаваемому модулю**

Группа: Z3300

Студент: Георгий Евгеньевич

Преподаватель: Михайлович Константин

1 Цель работы

- Изучить принципы проектирования синхронных цифровых устройств на языке описания аппаратуры и разработать реверсивный 8-разрядный двоичный счётчик с задаваемым модулем счёта, асинхронным сбросом и управлением направлением счёта.

2 Техническое задание

Необходимо разработать реверсивный 8-разрядный двоичный счётчик с задаваемым модулем счёта.

Счётчик должен функционировать от базового тактового сигнала частотой 50 МГц. Частота изменения значения счётчика должна составлять 1 Гц, для чего требуется реализовать делитель частоты.

Направление счёта определяется управляющим сигналом DIR:

- при $DIR = 0$ счёт выполняется вверх;
- при $DIR = 1$ счёт выполняется вниз.

При нажатии кнопки **RESET** должен выполняться асинхронный сброс счётчика в нулевое состояние независимо от тактового сигнала.

При нажатии кнопки **LOAD** в счётчик должен загружаться модуль счёта, поступающий на входы `INIT(7 downto 0)`.

Разрабатываемое устройство должно обеспечивать корректную работу во всех предусмотренных режимах функционирования.

3 Задачи, решаемые при выполнении работы

- Разработать VHDL-описание устройства.
- Выполнить моделирование в ModelSim.

4 Выполнение работы

4.1 Реализация на языке VHDL

Реализация счетчика приведена ниже:

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity updown_counter is
6 port (
7   clk    : in    std_logic;
8   res    : in    std_logic;
9   load   : in    std_logic;
10  dir    : in    std_logic;
11  init   : in    std_logic_vector(7 downto 0);
12
13  hex0   : out   std_logic_vector(7 downto 0);
14  hex1   : out   std_logic_vector(7 downto 0);
15  hex2   : out   std_logic_vector(7 downto 0);
16
17  cnt    : out   std_logic_vector(7 downto 0)
18 );
19 end updown_counter;
20
21 architecture rtl of updown_counter is
22
23   constant CLK_FREQ    : integer := 50_000_000;
24   constant TIMER_MAX   : integer := CLK_FREQ - 1;
25
26   signal timer_clk      : integer range 0 to TIMER_MAX := 0;
27   signal counter        : unsigned(7 downto 0) := (others =>
28     '0');
29
30   signal max_count      : unsigned(7 downto 0) := (others =>
31     '0');
32
33   signal hundreds      : unsigned(3 downto 0);
34   signal tens           : unsigned(3 downto 0);
35   signal ones           : unsigned(3 downto 0);
36
37   function digit_to_7seg(digit : unsigned(3 downto 0))
38   return std_logic_vector is
39   begin
40     case digit is
41       when "0000" => return "11000000"; -- 0
42       when "0001" => return "11111001"; -- 1
43       when "0010" => return "10100100"; -- 2
44       when "0011" => return "10110000"; -- 3
45       when "0100" => return "10011001"; -- 4
46       when "0101" => return "10010010"; -- 5
47       when "0110" => return "10000010"; -- 6
48       when "0111" => return "11111000"; -- 7
49       when "1000" => return "10000000"; -- 8
50       when "1001" => return "10010000"; -- 9
51       when others => return "11111111"; -- off
52     end case;
```

```

51 end function;
52
53 begin
54
55 process(clk, res)
56 begin
57     -- reset
58     if res = '0' then
59         timer_clk <= 0;
60         counter    <= (others => '0');
61
62     elsif rising_edge(clk) then
63         -- we can do one (load || update timer clk (
64             update counter))
65
66         -- load data
67         if load = '0' then
68             max_count <= (unsigned(init) - 1);
69
70             counter    <= (others => '0');
71             timer_clk <= 0;
72
73         -- update timer clk (update counter)
74         else
75             -- update timer clk if timer max
76             if timer_clk = TIMER_MAX then
77                 timer_clk <= 0;
78
79             -- update counter
80             if dir = '0' then
81                 if counter = max_count then
82                     counter <= (others => '0');
83                 else
84                     counter <= counter + 1;
85                 end if;
86             else
87                 if counter = 0 then
88                     counter <= max_count;
89                 else
90                     counter <= counter - 1;
91                 end if;
92             end if;
93
94             -- update timer clk is not timer max
95             else
96                 timer_clk <= timer_clk + 1;
97             end if;
98         end if;
99     end if;
100 end process;
101

```

```

102 -- set numbers
103 hundreds <= to_unsigned(to_integer(counter) / 100, 4);
104 tens      <= to_unsigned((to_integer(counter) / 10) mod 10,
105    4);
106
107 -- set led display
108 hex2 <= digit_to_7seg(hundreds);
109 hex1 <= digit_to_7seg(tens);
110 hex0 <= digit_to_7seg(ones);
111
112 -- set out current counter stage for tb
113 cnt <= std_logic_vector(counter);
114
115 end rtl;

```

После компиляции полученна следующая логическая схема:

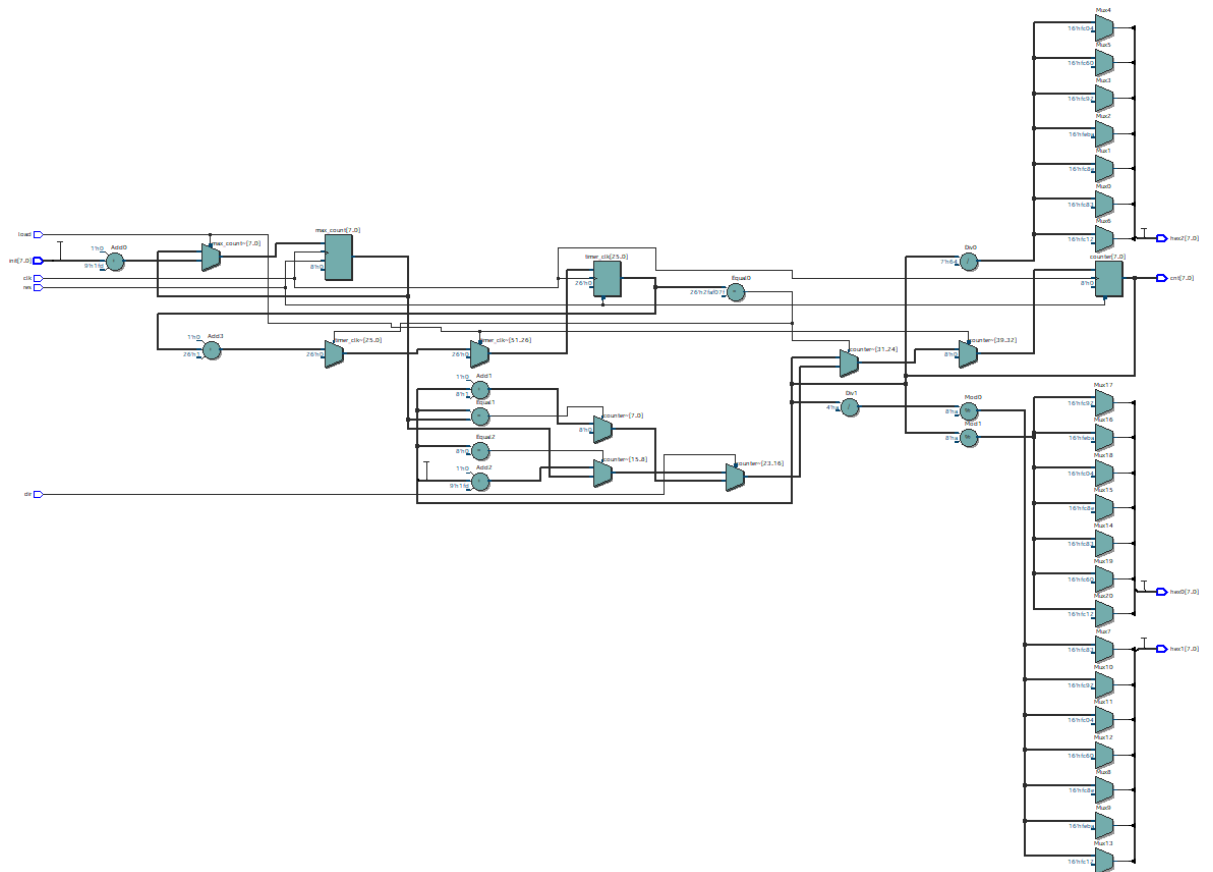


Рис. 1: Автоматически сгенерированная логическая схема, цельный вид

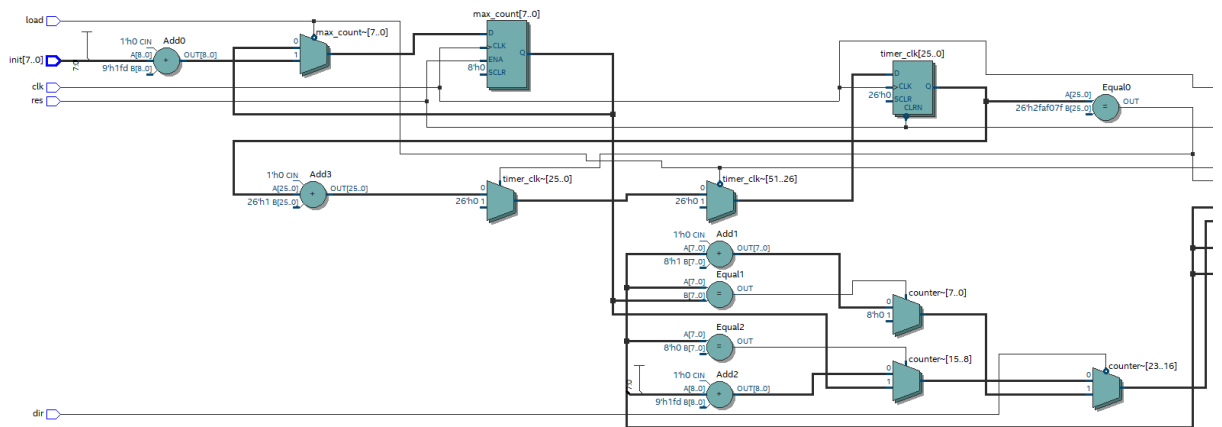


Рис. 2: Автоматически сгенерированная логическая схема, часть с триггерами

4.2 Моделирование в ModelSim

Приведем файлы test bench которые задают симуляцию логических схем в RTL Simulation

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity tb_updown_counter is
6 end tb_updown_counter;
7
8 architecture sim of tb_updown_counter is
9
10     constant CLK_PERIOD           : time := 20 ns;
11     constant CLK_PERIOD_int : integer := 20;
12     constant CLK_FREQ           : integer :=
13         10; --50_000_000;
14
15     signal clk           : std_logic
16         := '0';
17     signal res           : std_logic
18         := '1';
19     signal load          : std_logic
20         := '1';
21     signal dir           :
22         std_logic := '0';
23     signal init          :
24         std_logic_vector(7 downto 0) := (others => '0');
25
26     signal hex0          :
27         std_logic_vector(7 downto 0);
28     signal hex1          :
29         std_logic_vector(7 downto 0);
30
31 end architecture sim;

```

```

22         std_logic_vector(7 downto 0);
23         signal hex2
24             std_logic_vector(7 downto 0);
25
26         signal cnt
27             std_logic_vector(7 downto 0);
28
29 begin
30 uut: entity work.updown_counter
31     port map (
32         clk => clk,
33         res => res,
34         load => load,
35         dir => dir,
36         init => init,
37         hex0 => hex0,
38         hex1 => hex1,
39         hex2 => hex2,
40         cnt => cnt
41     );
42
43 clk_process : process
44 begin
45     for it in 0 to (2000*CLK_FREQ*CLK_PERIOD_int
46         ) loop
47         clk <= '0';
48         wait for CLK_PERIOD / 2;
49         clk <= '1';
50         wait for CLK_PERIOD / 2;
51     end loop;
52 end process;
53
54 monitor_process : process(clk)
55 begin
56     if rising_edge(clk) then
57         report "counter=" & integer'image(
58             to_integer(unsigned(cnt)));
59     end if;
60 end process;
61
62 stim_process : process
63 begin
64     -- reset
65     res <= '0';
66     wait for 100 ns;
67     res <= '1';
68
69     -- INIT = 0
70     init <= std_logic_vector(to_unsigned(0, 8));
71     load <= '0';

```

```

70     wait for CLK_PERIOD;
71     load <= '1';
72
73     wait for CLK_PERIOD*5*CLK_FREQ;
74
75     -- INIT = 255, count up
76     init <= std_logic_vector(to_unsigned(10, 8));
77     dir <= '0';
78     load <= '0';
79     wait for CLK_PERIOD;
80     load <= '1';
81
82     wait for CLK_PERIOD*12*CLK_FREQ;
83
84     -- INIT = 100, count up
85     init <= std_logic_vector(to_unsigned(15, 8));
86     dir <= '0';
87     load <= '0';
88     wait for CLK_PERIOD;
89     load <= '1';
90
91     wait for CLK_PERIOD*17*CLK_FREQ;
92
93     -- INIT = 50, count down
94     init <= std_logic_vector(to_unsigned(12, 8));
95     dir <= '1';
96     load <= '0';
97     wait for CLK_PERIOD;
98     load <= '1';
99
100    wait for CLK_PERIOD*15*CLK_FREQ;
101
102    -- again count up
103    dir <= '0';
104    wait for CLK_PERIOD*15*CLK_FREQ;
105
106    -- reset
107    res <= '0';
108    wait for 100 ns;
109    res <= '1';
110
111    wait for CLK_PERIOD*12*CLK_FREQ;
112
113    assert false report "Simulation finished" severity
        failure;
114    end process;
115
116    end sim;

```

Для моделирования зададим основную переменную 'CLK_FREQ = 5' чтобы на графике было видно как работает 'timer_clk' и 'timer_counter'. Далее представлены графики полученные в RTL Simulation

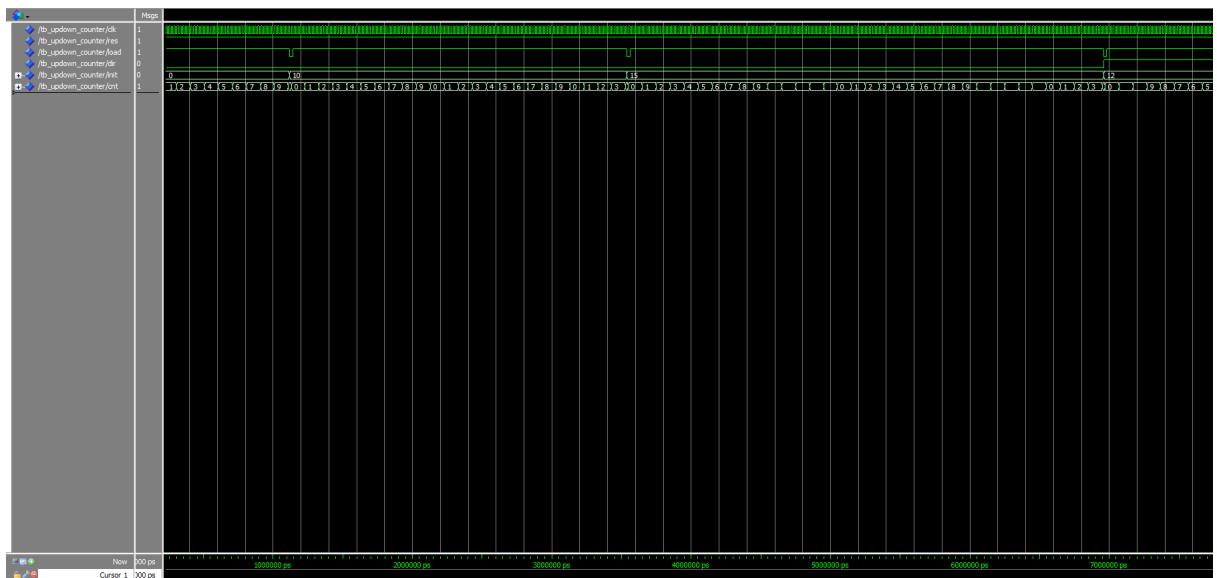


Рис. 3: Файл симуляция работы счетчика в начале

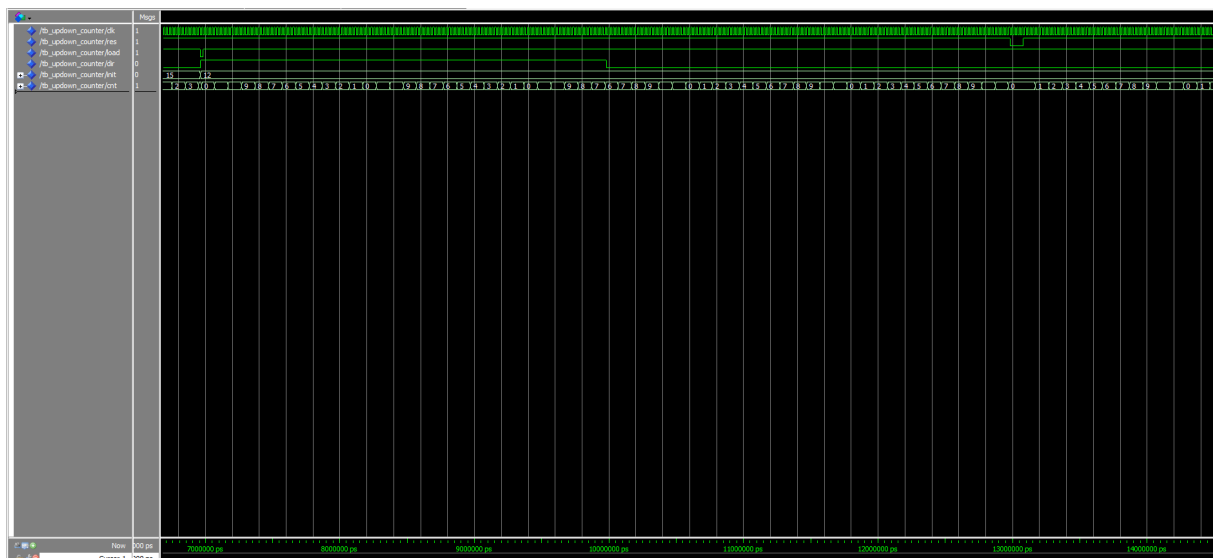


Рис. 4: Файл симуляция работы счетчика в конце

4.3 Сравнение результатов

Анализируя полученные графики в RTL Simulation, видим, что они эквивалентны. Что подтверждается на эксперименте, после загрузки в ПЛИС Altera MAX 10 FPGA на процессоре 10M50DAF484C7G